Qué son?

Las `funciones` o `métodos` (*una definición de función un poco mas especifica que veremos después*) nos permiten crear bloques de código reutilizables.

> Nos permiten crear grupos de instrucciones que podemos ejecutar luego con una sola línea cuantas veces queramos.

Ya hemos usado muchas funciones como `print` o `input`, pero ahora vamos a crear nuestras propias funciones.

Se crean usando la palabra `def` seguido de su *identificador (nombre),* un par de paréntesis y dos puntos:

> Para su identificador, usamos el mismo estándar que las variables: el `snake_case`.

Por ejemplo, puedo crear una función que llame un print para imprimir `Hello World` y `Mi primera funcion`. A esta función la llamaré `print_hello_world`:

```
1  def print_hello_world():
2    print("Hello World!")
3    print("Mi primera funcion")
```

Copiar

Si ejecutamos esto, no pasará nada.

Esto es porque solo estamos creando o *definiendo* la función, pero nunca la estamos ejecutando.

Para ejecutarla, o "llamarla", es exactamente igual que hacer un `print`: usando su *identificador* seguido de paréntesis. Es decir `print_hello_world()`:

```python
def print_hello_world():
  print("Hello World!")
  print("Mi primera funcion")


print_hello_world()
print_hello_world()
```

Copiar

(va a imprimirlo dos veces porque la estamos llamando dos veces)

```
Hello World!
Mi primera funcion
Hello World!
Mi primera funcion
```

Las funciones pueden tener cualquiera de las instrucciones que hemos estado usando hasta ahora.

Incluso pueden tener sus propias variables y llamar a otras funciones.

Es decir que podemos tomar cualquier código que hayamos escrito y convertirlo en una función.

Antes (sin funciones)

```python
worked_hours = int(input("Ingrese sus horas
hour_rate = int(input("Ingrese su tarifa po

salary = worked_hours * hour_rate

print(f'Su salario sera de {salary}')
```

Copiar

Después (convertido a función)

```python
def calculate_salary():
  worked_hours = int(input("Ingrese sus h
  hour_rate = int(input("Ingrese su tari
  salary = worked_hours * hour_rate
  print(f'Su salario sera de {salary}')

calculate_salary()
```

Copiar

Un código puede tener `n` cantidad de funciones distintas.

Y estas se pueden llamar entre sí.

Lo mejor es siempre definir las funciones arriba del archivo, separadas por 2 líneas en blanco.

Y ordenarlas en el orden en que se ejecutan.

Mal ordenado

Bien ordenado

```python
def function_2():
  print('Ejecutando 2')
main()

def main():
  function_1()
  function_2()
  function_3()

def function_1():
  print('Ejecutando 1')
def function_3():
  print('Ejecutando 3')
```

Copiar

```python
def function_1():
  print('Ejecutando 1')


def function_2():
  print('Ejecutando 2')


def function_3():
  print('Ejecutando 3')


def main():
  function_1()
  function_2()
  function_3()


main()
```

Copiar

Parámetros y Retornos

Todas las funciones tienen sus propios inputs y outputs (entradas y salidas).

Es decir que pueden recibir datos y devolver otros datos.

Parecido a una licuadora a la que le echas frutas y leche, y te devuelve un batido.

A sus inputs les llamamos `parámetros`.

A sus outputs les llamamos `retornos`.

Esta es la sintaxis de ambos **(y ya la veremos más a fondo)**:

```
def nombre(parameter1, parameter2, etc):
  # instruccion 1
  # instruccion 2
  # instruccion 3
  return output
```

Copiar

Parámetros

Al definir `parámetros` estamos definiendo *variables "placeholder" a las que les daremos un valor distinto cada vez que ejecutemos la función*.

Al ejecutarla, estos valores se establecen dentro de los paréntesis en el orden en que los definimos.

Por ejemplo, creemos una función que imprima 3 `parámetros`:

```
def print_parameters(parameter_1, parameter_2, parameter_3):
  print(f'This is parameter 1: {parameter_1}')
  print(f'This is parameter 2: {parameter_2}')
  print(f'This is parameter 3: {parameter_3}')


print_parameters(50, 'Hello', True)
```

Copiar



This is parameter 1: 50
This is parameter 2: Hello
This is parameter 3: True

Ahora intentemos ejecutarla varias veces dándole distintos valores a sus `parámetros`:

Copiar

```
def print_parameters(parameter_1, parameter_2, parameter_3):
  print(f'This is parameter 1: {parameter_1}')
  print(f'This is parameter 2: {parameter_2}')
  print(f'This is parameter 3: {parameter_3}')


print_parameters(50, 'Hello', 90)
print('-')
print_parameters([4, 5, 6], 'World', 'Inside')
print('-')
print_parameters('A', 'Function', True)
```

```
This is parameter 1: 50
This is parameter 2: Hello
This is parameter 3: 90
-
This is parameter 1: [4, 5, 6]
This is parameter 2: World
This is parameter 3: Inside
-
This is parameter 1: A
This is parameter 2: Function
This is parameter 3: True
```

Parámetros opcionales

Otro detalle útil es que podemos definir `parámetros opcionales` en las funciones.

Es decir, hacer que ciertos `parámetros` sean opcionales y tengan un valor por defecto en caso de que no se especifiquen a la hora de ejecutarlas.

Estos `parámetros` opcionales deben ir **después** de los requeridos.

Esto lo hacemos justo como al darle un valor a una variable.

```
1  def print_sum_of_numbers(number_a, number_b=5):
2    print(number_a + number_b)
3
4
5  print_sum_of_numbers(4)
```

Copiar

El number_a será 4, y el number_b será 5 porque no le dimos un valor.

9

Retornos

La palabra `return` nos permite decidir qué valor o valores serán el output de la función.

Pueden haber varios `return` en una función, pero **el primero que se ejecute es el que terminará la función**. Nada de lo que haya después de él se ejecutará.

Parecido a un `break` en un ciclo.

Estos outputs los podemos guardarlos en variables o usarlos como valores comunes y corrientes.

Por ejemplo, cuando guardamos el output de un `input` en una variable.

Por ejemplo, podemos crear una función que sume tres números y retorne su resultado:

```python
def sum_three_numbers(number1, number2, number3):
  return number1 + number2 + number3


result = sum_three_numbers(600, 700, 800)
print(result)
```

Copiar

O podemos crear una función que encuentre el máximo de 2 números y lo retorne:

```python
def get_max_of_two_numbers(number1, number2):
  if number1 > number2:
    return number1

  return number2


print(get_max_of_two_numbers(3, 7))
```

Copiar

```
7
13
6
```

Esta función tiene **2 parámetros**, `number1` y `number2`.

Si `number1` es mayor a `number2`, retorna number1.

El `return` hace que la función termine. Su output sale y el resto no se ejecuta. Por eso es que no es necesario el `else` en este caso. Sabemos que si la primera condición se cumple, se ejecuta el `return` y el segundo `return` nunca se ejecutará.

Si no es así, retorna `number2`.

Podemos notar es que estamos encadenando dos funciones, `print` y `get_max_of_two_numbers`.

Esto es porque ya sabemos que esa función va a retornar un valor.

Y una función ejecutada es igual a utilizar un valor o una variable.

Podemos usar llamados a funciones como parámetros para otras funciones, ya que el código se ejecutará de adentro hacia afuera.

Es exactamente lo mismo a hacer esto:

```python
def get_max_of_two_numbers(number1, number2):
  if number1 > number2:
    return number1

  return number2


result = get_max_of_two_numbers(4, 15)
print(result)
```

Copiar

15

Otras consideraciones importantes

Algunas funciones no tienen ni `parámetros`, ni `retornos`

Como las que vimos al inicio.

Algunas funciones solo `retornan` cosas, pero no tienen `parámetros`.

Algunas funciones solo tienen `parámetros`, pero no `retornan` nada.

Como el `print`. Se encarga de mostrar sus parámetros en pantalla, pero no los retorna.

```python
print(print('Primer print'))
```

Copiar

Primer print
None

Al igual que con ejemplo anterior, esto ultimo es igual a hacer:

```
1  result = print('Primer print')
2  print(result)
```

Copiar

```
Primer print
None
```

Nombramiento

Algo muy importante, al igual que con las variables, es darle a las funciones nombres **descriptivos y significativos.**

Esto con el objetivo de que cualquier persona que lea el código pueda entenderlo con facilidad.

Esto ultimo es la base de un código limpio, y es **sumamente importante en ambientes laborales** donde la colaboración es la clave.

Si al nombrar una función tenemos que usar "y", lo mejor es dividir esa función en dos funciones.

Lo mejor es usar **nombres verbales**. Es decir, incluir verbos en el nombre para saber qué acción especifica realiza la función.

Por ejemplo:

Si tenemos una función que *suma* dos números, llamarla `sum_two_numbers`.

Si tenemos una función que *genera* una lista de estudiantes, llamarla `generate_student_list`.

Usos Principales

El uso principal de las funciones es seguir el principio DRY del código limpio: Don't Repeat Yourself.

Si hay algún procedimiento o patrón que estamos haciendo más de una ves, **lo mejor es convertirlo en una función.**

Así mismo, cuando tenemos un algoritmo muy extenso, **lo mejor es dividirlo en funciones más pequeñas** para mejorar su legibilidad.

> Si al nombrar una función tenemos que usar "y", lo mejor es dividir esa función en dos funciones.

Por ejemplo, si tengo un código en el que calculo el promedio de notas de todos varios estudiantes y reviso si se eximieron:

Copiar

```
# Scores
juan_scores = {
  "spanish_score": 75,
  "science_score": 95,
  "history_score": 54,
}
sofia_scores = {
  "spanish_score": 64,
  "science_score": 56,
  "history_score": 98,
}
paul_scores = {
  "spanish_score": 72,
  "science_score": 75,
  "history_score": 79,
}

# Averages
juan_scores["average_score"] = (juan_scores["spanish_score"] + juan_scores["science_score
sofia_scores["average_score"] = (sofia_scores["spanish_score"] + sofia_scores["science_s
paul_scores["average_score"] = (paul_scores["spanish_score"] + paul_scores["science_score

juan_scores["is_exempted"] = juan_scores["average_score"] > 70
sofia_scores["is_exempted"] = sofia_scores["average_score"] > 70
paul_scores["is_exempted"] = paul_scores["average_score"] > 70
```

Podemos notar que ese calculo de `(spanish_score + science_score + history_score) / 3` se repite para cada estudiante.

Tambien estamos repitiendo la validación de si el `average_score > 70`.

Además puede ser confuso y repetitivo de leer ya que estamos haciendo varias cosas a la vez.

Lo mejor dado estas circunstancias es dividir esto en funciones reutilizables más pequeñas.

Copiar

```
def get_average_score(scores):
  return (scores["spanish_score"] + scores["science_score"] + scores["history_score"]) /


def is_student_exempted(scores):
  return scores["average_score"] > 70


# Scores
juan_scores = {
  "spanish_score": 75,
  "science_score": 95,
  "history_score": 54,
}
sofia_scores = {
  "spanish_score": 64,
  "science_score": 56,
  "history_score": 98,
}
paul_scores = {
  "spanish_score": 72,
  "science_score": 75,
  "history_score": 79,
}

# Averages
juan_scores["average_score"] = get_average_score(juan_scores)
sofia_scores["average_score"] = get_average_score(sofia_scores)
paul_scores["average_score"] = get_average_score(paul_scores)

```

```
juan_scores["is_exempted"] = is_student_exempted(juan_scores)
sofia_scores["is_exempted"] = is_student_exempted(sofia_scores)
paul_scores["is_exempted"] = is_student_exempted(paul_scores)
```

Incluso podemos hacer este código aun mejor convirtiendo estos diccionarios en una lista y haciendo un ciclo que haga lo mismo para todos:

Copiar

```
def get_average_score(scores):
  return (scores["spanish_score"] + scores["science_score"] + scores["history_score"]) /


def is_student_exempted(scores):
  return scores["average_score"] > 70


# Scores
students = [
  {
    "name": "Juan",
    "spanish_score": 75,
    "science_score": 95,
    "history_score": 54,
  },
  {
    "name": "Sofia",
    "spanish_score": 64,
    "science_score": 56,
    "history_score": 98,
  },
  {
    "name": "Paul",
    "spanish_score": 72,
    "science_score": 75,
    "history_score": 79,
  }
]

```

```
31  # Averages
32  for student in students:
33    student["average_score"] = get_average_score(student)
34    student["is_exempted"] = is_student_exempted(student)
35    print(student["name"], " is_exempted is ", student["is_exempted"])
```

Puntos Clave

Las funciones son bloques de código reutilizable que pueden aceptar y retornar datos.

Siempre es bueno definirlas arriba y dejar dos líneas en blanco entre cada una.

Siempre debemos de usar nombres descriptivos y verbales en nuestras funciones.

Siempre debemos de dividir nuestro código en funciones más pequeñas.